Delivery of Whole-blood to Wounded Soldier with Autonomous Drones

Written by Cody Hatfield

1. Abstract

Most pre-modern war deaths occurred off the battlefield, with disease killing far more people than weaponry. Anti-biotics and improved sanitation have all but eliminated disease as a concern in modern armies and most young soldiers can't even conceive of dying from cholera next to a latrine. Now, the leading cause of preventable death on the modern battlefield is overwhelmingly a result of blood hemorrhage. What's more, significant portions of these "bleeding out" casualties are entirely avoidable if the wounded individual receives a blood transfusion. These lives represent lost fathers, mothers, and children to the American people, and the preventability of their deaths render them all the more tragic.

For a critically wounded soldier on the battlefield, time is key. Even if the wound is technically survivable, they need to reach treatment quickly. While it may be possible for a medic to stop or significantly reduce bleeding in the field, there is unfortunately little that can be done once a soldier loses too much blood. When this volume of blood loss becomes too great, the soldier enters what is professionally known as the "golden hour", wherein said soldier must receive a blood transfusion within one hour or else they will die.

Past attempts were made to deploy blood bags with medics in the field. The blood would be stored in specialized containers that prevented expiration, and medics would be able to reduce casualties by stopping the bleeding before transfusing new blood into wounded soldiers. However, two problems arose:

- **Limits of Passive Thermoregulation**
    - Even with advanced insulating technology, the functional lifespan of the blood is around 18 hours.
    - Many units realistically could only get resupplied once a week.
- **Blood is Heavy**
    - Blood has around the same density as water.
    - Water is already very heavy to carry in large quantities.

Though the advances in insulating technology were a significant improvement, they were not enough to close the gap and see widespread adoption. The 18 hour time window is simply not long enough to last for the average medic on a frontline. This is particularly true if soldiers become encircled in dense urban terrain and must wait days or weeks for their comrades to break them out. Fixed wing aircraft cannot get close enough inside of an urban environment to deliver packages accurately and safely. The potential damage to a fixed-wing aircraft during active combat also poses a problem, as well as the potential for combatants to fire upon and destroy airdropped packages with parachutes.

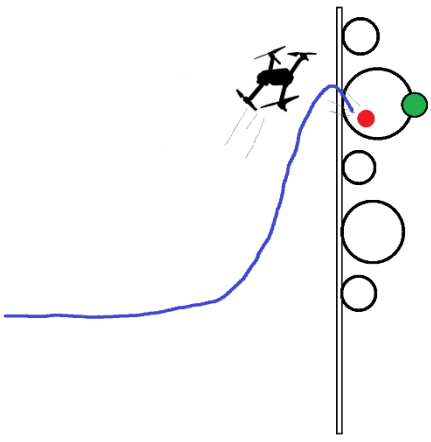
To address this clear need, our paper will present a novel copter-based delivery system wherein quadcopter drones will perform last mile delivery to geographic locations. These drones will actively use buildings as cover against air defense systems, and can throw the packages directly into open windows.

2. Entities

- Drone (Agent) – UAV
    - Our framework will assume a basic quad-copter design with outer body dimensions no larger than a meter.
    - Will be mounted with an actuator that drops small projectiles.
- Target Location
    - A physical location in space.
    - The projectile must be dropped on or near this target.
- Hoops
    - Has different sized variants
    - The projectile must be thrown through a chosen hoop.
    - Chosen hoop is marked with green circle.
- Hoop Holder
    - Static pole that holds the hoops in place.
- Projectile actuator
    - Robotic arm which holds and then drops the projectile.
- Projectile
    - The projectile which is dropped from the UAV.
    - Inert.

A tall pole holds a series of hoops. The drone will randomly be placed while facing the hoops. One of the hoops is the designated target window. The drone must fly to the target window, and throw the projectile through the window without hitting the hoops or going inside the window. Gusts of wind may occur, but only when a flag has been set to true.

3. Visualization



4. State Space

- Drone (Agent) – UAV
    - Step ID: Z
    - Position: R^3 = [x, y, z]
    - Forward Direction: R^3 = [x, y, z] (unit)
    - Velocity Direction: R^3 = [x', y', z'] (unit)
    - Speed: R
    - Angular Velocity: R^3 = [pitch', roll', yaw'], (world space)
    - Motor RPMs: R^{+4} = [front_right, front_left, back_right, back_left]
    - Actuator Signal: B
    - Target Direction: R^3 = [x, y, z] (unit)
    - Target Distance: R
    - Target Velocity Direction: R^3 = [x', y', z']
    - Target Speed: R
    - Throw Success: B
        - Target Hit is technically future information.
        - When a projectile is fired, and it goes near the target, the state which issued the actuator signal will have this value set to True, as well as all states after.
        - Initialized to False.
        - The drone will continue flying for a few frames.
    - Drone Collision: B
        - Initialized to False.
        - When the drone collides with the hoop or pole it is immediately is set to True.
        - Simulation resets immediately when this becomes True.
    - Step ID: Z
        - The step ID at which the projectile was dropped.
    - Projectile ID: Z
        - The PyBullet simulation ID number of the projectile which was dropped.
- Hoop
    - Position (Center): R^3 = [x, y, z]
- Hoop Holder
    - Position (Center): R^3 = [x, y, z]
- Target Marker
    - Position (Center): R^3 = [x, y, z]
    - Hoop pybullet Id: Z
- Projectile
    - Position: R^3 = [x, y, z]
    - Velocity: R^3 = [x', y', z']
    - Projectile Radius = R
        - The radius that determines how close the projectile needed to be to the target before it is considered a success.
    - Step ID: Z

        - The step ID at which the projectile was dropped.

5. Action Space

- Drone (Agent) – UAV Raptor
    - Motor RPMs: R^{+4} = [front_right, front_left, back_right, back_left]
    - Projectile Actuator: B

6. Start States

- Drone (Agent) – UAV
    - Position: The agent will spawn within a given set of positions inside the wall/window box
    - Angle: The agent will spawn within a given set of angles
    - Velocity: The agent will spawn within a given set of velocities
    - Angular Velocity: The agent will spawn within a given set of angular velocities
    - Actuator Signal: The agent will spawn with an actuator signal given
    - Target: The agent will spawn with a particular target
    - Target Hit: Initialized to False
- Hoop
    - Will be randomly placed on pole
- Target Marker
    - Position: The target will choose from a list of hoops and spawn on one.

7. Goal State

- Drone (Agent) – UAV
    - Throw Success: True

8. Reward Map

- Actuator Activation
    - Reward is scaled based on the closest distance the projectile has with the target.
    - Negative reward if closest distance is above a threshold
    - Positive reward if closest distance is below threshold.
    - 0 Reward if at the threshold.
    - All states where Target Hit is True all receive the positive reward.
- Altitude
    - Negative reward if altitude is not within a given set.
- Target Distance
    - Negative reward if distance to the target is too small.
- Drone Collision
    - Negative reward if the drone collides with a hoop or pole.

9. Scenario Variations

- Hoops may all be of one size
- Hoops may all be of random size